

Homework 3: Decision Trees and K-Nearest-Neighbor

Your solutions to theoretical questions should be done in Markdown and Latex directly below the associated question.

Your solutions to computational questions should include any specified Python code and results as well as written commentary on your conclusions.

Remember that while you can discuss the problems with your classmates, **you must write all code and solutions on your own**. Please read through the course Academic Honesty Policy [in the syllabus](#).

Some problems with code may be autograded. If we provide a function API, **do not** change it. If we do not provide a function API, then you're free to structure your code however you like.

Submission instructions:

Please submit **two** things to the Gradescope "Homework 3" assignment: this Jupyter notebook **and** a PDF of this notebook. Do not compress it using tar, zip, etc.

Name: Daniela Avayu

```
In [1]: import math
import pickle
import gzip
import numpy as np
import pandas as pd
import matplotlib.pyplot as plt
import seaborn as sns
from sklearn.model_selection import train_test_split
%matplotlib inline
```

[60 points] Problem 1 - Decision tree

Consider the problem of predicting whether a person has a college degree based on age, salary, and Chicago residency. The dataset looks like the following.

Age	Salary	Chicago Residency	College degree
27	41,000	Yes	Yes
61	52,000	No	No
23	24,000	Yes	No
29	77,000	Yes	Yes
32	48,000	No	Yes
57	120,000	Yes	Yes

Age	Salary	Chicago Residency	College degree
22	38,000	Yes	Yes
41	45,000	Yes	No
53	26,000	No	No
48	65,000	Yes	Yes

Part A [5 points]: Convert the above table to data. Two variables should be created:

1. X is a 10×3 matrix that contains the data from columns 0, 1, and 2. Chicago residency is represented by 1 (yes) and 0 (no).
2. y contains the labels (college degree), 1 (yes) and 0 (no).

```
In [2]: data = {
    "age": [27,61,23,29,32,57,22,41,53,48],
    "salary": [41000,52000,24000,77000,48000,120000,38000,45000,26000,65000],
    "chicago_residency": [1,0,1,1,0,1,1,1,0,1],
    "college_degree": [1,0,0,1,1,1,1,0,0,1]
}

df = pd.DataFrame(data)

x = df[["age", "salary", "chicago_residency"]].values
y = df["college_degree"].values

print(df)
```

```
   age  salary  chicago_residency  college_degree
0   27   41000                   1                 1
1   61   52000                   0                 0
2   23   24000                   1                 0
3   29   77000                   1                 1
4   32   48000                   0                 1
5   57  120000                   1                 1
6   22   38000                   1                 1
7   41   45000                   1                 0
8   53   26000                   0                 0
9   48   65000                   1                 1
```

Part B [10 points]: Criteria for choosing a feature to split.

[4 points] We start with no splitting. Assuming that our algorithm is deterministic, what is the smallest number of mistakes we can make if we do not use any of the features and what is the algorithm? (**Write your answer in the Markdown cell below.**)

Since we are not using any features, the best deterministic algorithm is to always predict the majority class in the training data. In this case, there are 6 people with college_degree=1 and 4 people with college_degree=0, so we should always predict college_degree=1. This makes mistakes on the 4 people whose true label is 0.

[6 points] We start by considering the variable *Chicago residency*. The first criteria is based on the number of mistakes.

How many mistakes will we make if we split based on Chicago residency? **(Answer below by finishing the code.)**

```
In [3]: def get_error_in_leaf(y, ids):
        """
        Returns the errors in a leaf node of a decision tree.
        This function can be used to answer the previous question automatically.

        :@param y: all labels
        :@param ids: the subset of indexes in the leaf node
        """
        labels = y[ids]
        num_ones = np.sum(labels == 1)
        num_zeros = np.sum(labels == 0)

        return min(num_ones, num_zeros)

def error_criteria(y, root, left_child, right_child):
    """
    Returns the number of errors if we split the root into the left child and the right child.

    :@param y: all labels
    :@param root: indexes of all the data points in the root
    :@param left_child: the subset of indexes in the left child
    :@param right_child: the subset of indexes in the right child
    """

    left_errors = get_error_in_leaf(y, left_child)
    right_errors = get_error_in_leaf(y, right_child)

    return left_errors + right_errors

def value_split_binary_feature(x, y, fid, root, criteria_func):

    root = root.tolist()
    left_child = [i for i in root if x[i, fid] == 0]
    right_child = [i for i in root if x[i, fid] == 1]
    return criteria_func(y, root, left_child, right_child)

# Chicago residency should correspond to the third column in your data x
fid = 2
root = np.array(list(range(len(y)))) # root includes all data points
mistakes = value_split_binary_feature(x, y, fid, root, error_criteria)
```

Splitting on Chicago residency gives 3 mistakes, which is better than the original no-split model with 4 mistakes.

Chicago residency = 0: labels are [0, 1, 0] best prediction is 0 mistakes = 1

Chicago residency = 1: labels are [1, 0, 1, 1, 1, 0, 1] best prediction is 1 mistakes = 2

This cell is for grading purposes only; please ignore

Part C [20 points] In class, we used Gini impurity measure to split data at each node of a decision tree. Alternatively, we can use entropy and information gain to split the data. In this part, you will manually determine the first split in a decision tree. Please do not use code in your calculations for part C.

Information gain is the difference between the impurity at the parent node and a weighted average of the impurity of the its children. That is, information gain tell you how much better you can predict the output after splitting the data on a feature.

Information gain is given by:

$$IG = H(\text{parent}) - \sum_k \frac{D_k}{D} H(\text{child}_k)$$

where H is entropy.

[4 points] Write the equations necessary to compute entropy and information gain if we split data D into D_1 and D_2 (two children). Hint: See the lecture slides for a mathematical definition of entropy.

Entropy is defined as:

$$H(D) = - \sum_{c \in \{0,1\}} p_c \log_2(p_c)$$

where (p_c) is the proportion of observations in dataset (D) with class label (c).

For binary labels, this can be written as:

$$H(D) = -p_0 \log_2(p_0) - p_1 \log_2(p_1)$$

- Information gain = old impurity - new weighted impurity

If we split (D) into two children, (D_1) and (D_2), then information gain is:

$$IG(D, D_1, D_2) = H(D) - \left(\frac{|D_1|}{|D|} H(D_1) + \frac{|D_2|}{|D|} H(D_2) \right)$$

- If information gain is high, the split was useful. If information gain is low, the split did not help much.

[4 points] What is the entropy for College Degree? Please show your work. Were you expecting a result around this number? Why?

For College Degree, the labels are: 1, 0, 0, 1, 1, 1, 1, 0, 0, 1. There are 10 observations total: 6 with college degree and 4 without.

So,

$$p_1 = \frac{6}{10} = 0.6$$

$$p_0 = \frac{4}{10} = 0.4$$

Entropy is:

$$H(D) = -p_0 \log_2(p_0) - p_1 \log_2(p_1)$$

Plugging in the values,

$$H(D) = -0.4 \log_2(0.4) - 0.6 \log_2(0.6)$$

$$H(D) = -0.4(-1.322) - 0.6(-0.737)$$

$$H(D) = 0.529 + 0.442$$

$$H(D) \approx 0.971$$

So, the entropy for college degree is:

$$\boxed{0.971}$$

Yes, I would expect a result around this number because the labels are relatively mixed: 6 yes and 4 no. Entropy is highest when the classes are perfectly balanced, like 5 and 5, where entropy would be 1. Since 6 and 4 is close to balanced, the entropy is close to 1.

[4 points] What is the information gain for College Degree if you split the observations on the Chicago Residency attribute? Please show your work.

Chicago Residency = 0 Rows with labels = 0, 1, 0

$$p_0 = \frac{2}{3}, \quad p_1 = \frac{1}{3}$$

$$H(D_0) = -\frac{2}{3} \log_2\left(\frac{2}{3}\right) - \frac{1}{3} \log_2\left(\frac{1}{3}\right)$$

$$H(D_0) \approx 0.918$$

Chicago Residency = 1 Rows with labels = 1, 0, 1, 1, 1, 0, 1

$$p_1 = \frac{5}{7}, \quad p_0 = \frac{2}{7}$$

$$H(D_1) = -\frac{5}{7} \log_2\left(\frac{5}{7}\right) - \frac{2}{7} \log_2\left(\frac{2}{7}\right)$$

$$H(D_1) \approx 0.863$$

The parent entropy from the previous question was:

$$H(D) \approx 0.971$$

Computing the weighted child entropy,

$$\begin{aligned}
 & \frac{3}{10}H(D_0) + \frac{7}{10}H(D_1) \\
 &= \frac{3}{10}(0.918) + \frac{7}{10}(0.863) \\
 &= 0.275 + 0.604 \\
 &= 0.879
 \end{aligned}$$

Information gain is:

$$\begin{aligned}
 IG &= H(D) - \left[\frac{3}{10}H(D_0) + \frac{7}{10}H(D_1) \right] \\
 IG &= 0.971 - 0.879 \\
 IG &\approx 0.092
 \end{aligned}$$

The information gain for splitting on Chicago Residency is:

$$\boxed{0.092}$$

[4 points] One way to deal with continuous (or ordinal) features like Age and Salary is to define binary features based on thresholding. For example, you might convert ages to 0 if Age is less than or equal to 50 and 1 otherwise. What is the information gain for College Degree if we split the observations based on the Salary attribute with a threshold of \$50,000? Please show your work.

(Later in this problem, we will write code to determine whether a number other than 50,000 might be the optimal threshold for Salary. For now, just assume that a threshold of 50,000 is optimal.)

For a salary threshold of \$50,000, we split the data into two groups: Salary $\leq$ 50,000 Salary $>$ 50,000

Parent entropy (from previous exercise):

$$H(D) \approx 0.971$$

For the group where Salary $\leq$ 50,000, the college degree labels are: 1, 0, 1, 1, 0, 0 There are 6 observations.

$$p_1 = \frac{3}{6} = 0.5$$

$$p_0 = \frac{3}{6} = 0.5$$

Entropy is:

$$H(D_1) = -0.5 \log_2(0.5) - 0.5 \log_2(0.5)$$

$$H(D_1) = 1$$

For the group where Salary > 50,000, the college degree labels are: 0, 1, 1, 1 There a 4 observations.

$$p_1 = \frac{3}{4}$$

$$p_0 = \frac{1}{4}$$

Entropy is:

$$H(D_2) = -\frac{3}{4}\log_2\left(\frac{3}{4}\right) - \frac{1}{4}\log_2\left(\frac{1}{4}\right)$$

$$H(D_2) \approx 0.811$$

The weighted entropy after split is:

$$\frac{6}{10}H(D_1) + \frac{4}{10}H(D_2)$$

$$= \frac{6}{10}(1) + \frac{4}{10}(0.811)$$

$$= 0.600 + 0.324$$

$$= 0.924$$

$$IG = H(D) - \left[\frac{6}{10}H(D_1) + \frac{4}{10}H(D_2) \right]$$

$$IG = 0.971 - 0.924$$

$$IG \approx 0.047$$

So the information gain after splitting on salary with 50,000 threshold is:

$$\boxed{0.047}$$

[4 points] Based on the information gain calculations, should the first split in our decision tree be on Chicago Residency or on Salary with a \$50,000 threshold? Is this the answer you expected based on the data counts? Why or why not?

Based on the information gain calculations, the first split should be on Chicago Residency because it has a higher information gain than Salary with a \$50,000 threshold. We found that the information gain for Chicago Residency is about (0.092), while the information gain for Salary is about (0.047). Since decision trees choose the split that reduces uncertainty the most, Chicago Residency is the better first split. This makes sense based on the data counts because the Salary split creates one group that is perfectly mixed, with 3 people having a college degree and 3 people not having one, so that split does not help prediction very much. Chicago Residency still has mixed groups too, but they are slightly more useful for separating the college degree labels.

Part D [10 points] Now we will write a function for computing information gain. Use log base 2 for entropy computation.

```

In [4]: def entropy(y, ids):
        """
        Returns the entropy in the labels for the data points in ids.

        :@param y: all labels
        :@param ids: the indexes of data points
        """
        if len(ids) == 0: # deal with corner case when there is no data point.
            return 0

        labels = y[ids]
        # Count how many selected labels are 0/1.
        num_zeros = np.sum(labels == 0)
        num_ones = np.sum(labels == 1)

        # Compute the proportion of labels that are 0/1.
        p0 = num_zeros / len(ids)
        p1 = num_ones / len(ids)

        # Start entropy at 0, then add the entropy contribution from each class.
        entropy_value = 0

        # Only include this term if p0 is greater than 0.
        if p0 > 0:
            entropy_value -= p0 * np.log2(p0)

        if p1 > 0:
            entropy_value -= p1 * np.log2(p1)

        return entropy_value

def information_gain_criteria(y, root, left_child, right_child):
    """
    Returns the information gain by splitting root into left child and right child.

    :@param y: all labels
    :@param root: indexes of all the data points in the root
    :@param left_child: the subset of indexes in the left child
    :@param right_child: the subset of indexes in the right child
    """

    # Compute the entropy before splitting.
    parent_entropy = entropy(y, root)

    # Compute the fraction of the data that goes to the left/right child.
    left_weight = len(left_child) / len(root)
    right_weight = len(right_child) / len(root)

    # Compute the weighted average entropy after the split.
    # Larger child nodes count more than smaller child nodes.
    weighted_child_entropy = (
        left_weight * entropy(y, left_child)

```



```

        + right_weight * entropy(y, right_child)
    )

    # Information gain is the decrease in entropy after splitting.
    information_gain = parent_entropy - weighted_child_entropy

    return information_gain

fid = 2
root = np.array(list(range(len(y)))) # root includes all data points
info_gain = value_split_binary_feature(x, y, fid, root, information_gain_criteria)

```

In [5]: *# This cell is for grading purposes only; please ignore*

Part E [15 points]: Deal with continuous features.

[10 points] Complete the following code:

```

In [6]: def value_split_continuous_feature(x, y, fid, root, criteria_func=information_gain_criteria):
        """
        Return the best value and its corresponding threshold by splitting based on a continuous feature.

        :@param x: all feature values
        :@param y: all labels
        :@param fid: feature id to split the tree based on
        :@param root: indexes of all the data points in the root
        :@param criteria_func: the splitting criteria function
        """
        best_value, best_thres = 0, 0

        # Convert root to a list so we can loop through the row indexes easily.
        root = root.tolist()

        # Get all unique values of this feature among the data points in root.
        # These are the possible thresholds we will try.
        thresholds = np.unique(x[root, fid])

        # Try each possible threshold.
        for thres in thresholds:

            # Put observations with feature value <= threshold in the left/right child.
            left_child = [i for i in root if x[i, fid] <= thres]
            right_child = [i for i in root if x[i, fid] > thres]

            # Compute how good this split is using the given criteria function.
            # For this problem, the criteria is information gain.
            value = criteria_func(y, root, left_child, right_child)

            # If this threshold gives a better value than the best one so far,
            # save both the value and the threshold.
            if value > best_value:

```

```

        best_value = value
        best_thres = thres

    return best_value, best_thres

root = np.array(range(len(y))) # root includes all data points
fid = 0
age_value, age_thres = value_split_continuous_feature(x, y, fid, root, information_gain_criteria)
fid = 1
salary_value, salary_thres = value_split_continuous_feature(x, y, fid, root, information_gain_criteria)

```

In [7]: *# This cell is for grading purposes only; please ignore*

In [8]: *# This cell is for grading purposes only; please ignore*

[5 points] Based on the current information gain by splitting different features, if we build a decision stump (decision tree with depth 1) greedily, which feature should we choose? Why? **Write down your answer in the Markdown cell below.**

```

In [9]: root = np.array(range(len(y)))

# Chicago Residency
fid = 2
chicago_value = value_split_binary_feature(
    x, y, fid, root, information_gain_criteria
)

# Age
fid = 0
age_value, age_thres = value_split_continuous_feature(
    x, y, fid, root, information_gain_criteria
)

# Salary
fid = 1
salary_value, salary_thres = value_split_continuous_feature(
    x, y, fid, root, information_gain_criteria
)

print("Chicago Residency information gain:", chicago_value)
print("Age information gain:", age_value)
print("Salary information gain:", salary_value)

```

Chicago Residency information gain: 0.0912774462416801

Age information gain: 0.1444843438056279

Salary information gain: 0.3219280948873623

Age information gain: 0.144 Salary information gain: 0.322 Chicago Residency information gain: 0.091

So the decision stump should split on Salary because it has the highest information gain.

Extra credit [5 points]: You now have all the ingredients to build a decision tree recursively. You can build a decision tree of depth two and report its classification error on the training data and the tree.

```
In [10]: class LeafNode:
    """
    Class for leaf nodes in the decision tree
    """

    def __init__(self, label, count, total):
        """
        :@param label: label of the leaf node
        :@param count: number of data points with class 'label' falling in this leaf
        :@param total: number of datapoints of any label falling in this leaf
        """
        self.label = label
        self.count = count
        self.total = total

    def predict(self, x):
        """
        Return predictions for features x

        :@param x: feature values
        """
        # YOUR CODE HERE
        raise NotImplementedError()

    def display(self, feat_names, out_str, depth=0):
        """
        Display contents of a leaf node
        """
        prefix = '\t'*depth
        error = 1.0 - self.count / float(self.total)
        out_str += f'{prefix}leaf: label={self.label}, error={error} ({self.count}/{self.total} correct)\n'
        return out_str

class TreeNode:
    """
    Class for internal (non-leaf) nodes in the decision tree
    """

    def __init__(self, feat_id, feat_val):
        """
        :@param feat_id: index of the feature that this node splits on
        :@param feat_val: threshold for the feature that this node splits on
        """
        self.feat_id = feat_id
        self.feat_val = feat_val
        self.left = None
        self.right = None

    def split(self, x, root):
```

```

    """
    Given the datapoints falling into current node, return two arrays of indices in x corresponding to the
    left and right subtree

    :@param x: all feature values
    :@param root: indexes of all the data points in the current node
    """
    root = np.array(root)
    # YOUR CODE HERE
    raise NotImplementedError()

def predict(self, x):
    """
    Return an array of predictions for given 'x' for the current node

    :@param x: datapoints
    """
    assert self.left is not None and self.right is not None, 'predict called before fit'
    # YOUR CODE HERE
    raise NotImplementedError()

def display(self, feat_names, out_str, depth=0):
    """
    Display contents of a non-leaf node
    """
    prefix = '\t'*depth
    out_str += f'{prefix}{{feat_names[self.feats_id]}}\n'
    out_str += f'{prefix}x <= {self.feats_val}\n'
    out_str = self.left.display(feat_names, out_str, depth=depth+1)
    out_str += f'{prefix}x > {self.feats_val}\n'
    out_str = self.right.display(feat_names, out_str, depth=depth+1)
    return out_str

class DecisionTree:
    """
    Class for the decision tree
    """
    def __init__(self, max_depth=1, criteria_func=information_gain_criteria, binary_feats_ids=[]):
        """
        :@param max_depth: Maximum depth that a decision tree can take
        :@param criteria_func: criteria function to split features
        :@param binary_feats_id: list of indexes of binary features
        """
        self.max_depth = max_depth
        self.criteria_func = criteria_func
        self.binary_feats_ids = binary_feats_ids
        self.root = None
        self.x = None
        self.y = None

    def fit(self, x, y):
        """

```

```

    Fit a tree to the given dataset using a helper function
    """
    self.x = x
    self.y = y
    self.root = self.fit_helper(np.array(list(range(self.x.shape[0]))))

def fit_helper(self, root, depth=1):
    """
    Recursive helper function for fitting a decision tree
    Returns a node (can be either LeafNode or TreeNode)

    :@param root: array of indices of datapoints which fall into the current node
    :@param depth: current depth of the tree being built
    """

    """
    Strategy:
    1. If current partition is pure i.e. labels corresponding to all indices in root are the same
       OR the maximum depth has been reached, stop building the tree and return a LeafNode
    2. If not, find out the best feature to split on along with the threshold, create a TreeNode and
       recursively call fit_helper on the two splits (You can assume the threshold for a binary feature
       to be 0.5). Finally, return the current node
    """

    # YOUR CODE HERE
    raise NotImplementedError()

def predict(self, x):
    """
    Return predictions for a given dataset
    """
    assert self.root is not None, 'fit not yet called'
    # YOUR CODE HERE
    raise NotImplementedError()

def display(self, feat_names):
    assert self.root is not None, 'fit not yet called'
    out_str = ""
    out_str = self.root.display(feat_names, out_str)
    return out_str

```

```

In [11]: # This cell is for grading purposes only; please ignore
from sklearn.tree import DecisionTreeClassifier
from sklearn.metrics import accuracy_score

```

[40 points] Problem 2 - KNN for Crime Prediction

In this problem, you will implement a K-Nearest-Neighbors framework to predict the type of crime reported based on an incident's location.

We will use the [reported crime data for 2021](#) from the Chicago Open Data Portal (we provided this on Canvas, but want you to know where the source was).

Part A [5 points]: Load the data. We will only use three columns of the dataset: Primary Type, Latitude, and Longitude.

- Be sure to drop any observations that are missing latitude and/or longitude.
- To reduce run time, only keep incidents reported as one of the the four most common crime types ('THEFT', 'BATTERY', 'CRIMINAL DAMAGE', 'ASSAULT').
- Use the `train_test_split` function from Scikit-Learn to split the data into training and validation sets. Set `random_state=123` and `test_size=0.2`.
- Finally, explore the training and validation sets and answer the following questions:
 - How many total observations are in the training set?
 - How many total observations are in the validation set?

```
In [12]: # Write code for answering the questions in Part A and then put your answer in the Markdown cell below.
# Make sure to set each of the variables below to the correct value. Do not rename the variables.
N_training_examples = None
N_validation_examples = None

# Select only the columns we need from the full crime dataset.
crime = pd.read_csv("crime-2.csv")

# Keep only Primary Type, Latitude, and Longitude.
crime = crime[["Primary Type", "Latitude", "Longitude"]]

# Drop rows where Latitude or Longitude is missing.
crime = crime.dropna(subset=["Latitude", "Longitude"])

# Keep only the four most common crime types listed in the instructions.
crime_types = ["THEFT", "BATTERY", "CRIMINAL DAMAGE", "ASSAULT"]
crime = crime[crime["Primary Type"].isin(crime_types)]

# Create the feature matrix X using Latitude and Longitude.
X = crime[["Latitude", "Longitude"]]

# Create the label vector y using Primary Type.
y = crime["Primary Type"]

# Split the data into training and validation sets.
# test_size=0.2 means 20% validation and 80% training.
# random_state=123 makes the split reproducible.
X_train, X_val, y_train, y_val = train_test_split(
    X,
    y,
    test_size=0.2,
    random_state=123
)

# Print the number of observations in each set.
print("Training observations:", len(X_train))
print("Validation observations:", len(X_val))
```

Training observations: 99751
Validation observations: 24938

In [13]: *# This cell is for grading purposes only; please ignore*

- How many total observations are in the training set? 99751
- How many total observations are in the validation set? 24938

Optional: If you'd like to run the following code chunk to render a map of the incidents, you will need to download the [neighborhood boundaries data](#) from the Chicago Data Portal (again, we have provided a version on Canvas).

```
In [14]: # Import library
import geopandas as gpd

# Load boundaries
file = open('boundaries.geojson')
boundaries = gpd.read_file(file)

# Convert full crime dataset from pandas to geopandas
crime_gdf = gpd.GeoDataFrame(crime_df, geometry=gpd.points_from_xy(crime_df['Longitude'], crime_df['Latitude']),
                             crs='EPSG:4326')

# Make plot
fig, ax = plt.subplots(1, figsize=(12, 16))
boundaries.plot(ax=ax, color='white', edgecolor='black')
colors = {'THEFT' : 'red', 'BATTERY' : 'blue', 'CRIMINAL DAMAGE' : 'yellow', 'ASSAULT' : 'gray'}
grouped = crime_gdf.groupby('Primary Type', sort=True)
for key in colors:
    group = grouped.get_group(key)
    group.plot(ax=ax, label=key, color=colors[key], markersize=0.3)
ax.legend(markerscale=10)
ax.axis('off')
ax.set_title('Reported Incidents in Chicago, 2021', fontdict={'fontsize': '25', 'fontweight' : '3'});
```

```
-----
ModuleNotFoundError                                Traceback (most recent call last)
Cell In[14], line 2
      1 # Import library
----> 2 import geopandas as gpd
      3
      4 # Load boundaries
      5 file = open('boundaries.geojson')

ModuleNotFoundError: No module named 'geopandas'
```

```
In [15]: class KNN:
        """
        Class to store data for regression problems
        """
        def __init__(self, X_train, y_train, K=5, distance_weighted=False):
```

```

"""
Creates a kNN instance

:param X_train: Training data input in 2D ndarray
:param y_train: Training data output in 1D ndarray
:param K: The number of nearest points to consider in classification
:param distance_weighted: Bool indicating whether to use distance weighting
"""

# Import and build the BallTree on training features
from sklearn.neighbors import BallTree
self.balltree = BallTree(X_train)

# Cache training labels and parameter K
self.y_train = y_train
self.K = K

# Boolean flag indicating whether to do distance weighting
self.distance_weighted = distance_weighted

def majority(self, neighbor_indices, neighbor_distances=None):
    """
    Given indices of nearest neighbors in training set, return the majority label.
    Break ties by considering 1 fewer neighbor until a clear winner is found.

    :param neighbor_indices: The indices of the K nearest neighbors in self.X_train
    :param neighbor_distances: Corresponding distances from query point to K nearest neighbors.
    """

    # If only one neighbor, return label of that neighbor
    if len(neighbor_indices) == 1:
        return self.y_train[neighbor_indices[0]]

    # If no distances provided, set to ones
    if neighbor_distances is None:
        neighbor_distances = np.ones(len(neighbor_indices))

    # Get the labels of the current nearest neighbors.
    neighbor_labels = self.y_train[neighbor_indices]

    # Count how many times each label appears.
    labels, counts = np.unique(neighbor_labels, return_counts=True)

    # Find the largest vote count.
    max_count = np.max(counts)

    # Find all labels that received the largest vote count.
    winners = labels[counts == max_count]

    # If there is exactly one winner, return it.
    if len(winners) == 1:
        return winners[0]

```



```

        # If there is a tie, remove the farthest neighbor and try again.
        return self.majority(neighbor_indices[:-1], neighbor_distances[:-1])

def classify(self, x):
    """
    Given a query point, return the predicted label

    :param x: a query point stored as an ndarray
    """
    # BallTree expects a 2D array of query points.
    # x.reshape(1, -1) turns one query point into shape (1, p).
    distances, indices = self.balltree.query(
        x.reshape(1, -1),
        k=self.K
    )

    # BallTree returns arrays with shape (1, K), so we take [0]
    # to get the K distances and K indices for this one query point.
    neighbor_distances = distances[0]
    neighbor_indices = indices[0]

    # Return the majority label among the nearest neighbors.
    return self.majority(neighbor_indices, neighbor_distances)

def predict(self, X):
    """
    Given an ndarray of query points, return yhat, an ndarray of predictions

    :param X: an (m x p) dimension ndarray of points to predict labels for
    """
    # Classify each row of X and collect the predictions.
    predictions = [self.classify(x) for x in X]

    # Return predictions as a numpy array.
    return np.array(predictions)

```

Part B [10 points]: Modify the class above to implement an Unweighted KNN classifier. There are three methods that you need to complete:

- **predict** : Given an $m \times p$ matrix of validation data with m examples each with p features, return a length- m vector of predicted labels by calling the **classify** function on each example.
- **classify** : Given a single query example with p features, return its predicted class label as a string using KNN by calling the **majority** function.
- **majority** : Given an array of indices into the training set corresponding to the K training examples that are nearest to the query point, return the majority label as a string. If there is a tie for the majority label using K nearest neighbors, reduce K by 1 and try again. Continue reducing K until there is a winning label.

Notes:

- Don't even think about implementing nearest-neighbor search or any distance metrics yourself. Instead, go read the documentation for Scikit-Learn's [BallTree](#) object. You will find that its implemented [query](#) method can do most of the heavy lifting for you.
- **Do not** use Scikit-Learn's KNeighborsClassifier in this problem. We're implementing this ourselves.
- You don't need to worry about the `distance_weighted` flag until **Part C**, but we recommend reading ahead a bit. It might be good to think about your implementation of **Part C** before implementing **Part B**.
- When you think you're done, execute the following cell to run 4 unit tests.

```
In [ ]: from tests import tests
        tests.run_test_suite('prob 2A', KNN)
```

```
test1NNclassify (tests.tests.TestUnweightedKNN.test1NNclassify)
test 1NN ... ok
test2NNclassify (tests.tests.TestUnweightedKNN.test2NNclassify)
test 2NN. Checks tie-breaking. ... ok
test3NNclassify (tests.tests.TestUnweightedKNN.test3NNclassify)
test 3NN ... ok
test3NNpredict (tests.tests.TestUnweightedKNN.test3NNpredict)
test 3NN prediction ... ok
```

```
-----
Ran 4 tests in 0.032s
```

```
OK
```

Part C [5 points]: Modify the `KNN` class to perform the distance-weighted KNN classification. The so-called Distance-Weighted KNN classifier assigns weights to the nearest-neighbor training examples proportional to the inverse-distance from the training example to the query point. Classification is performed by summing the weights associated with each class and predicting the class with the highest weighted-majority vote. Mathematically we might describe the weighted-vote for a class c as

$$\text{Weighted-Vote}(c) = \sum_{i \in \mathcal{N}_K} I(y_i = c) \times \frac{1}{\|\mathbf{x}_i - \mathbf{x}\|}$$

A word of caution: it's certainly possible that a query point could be distance 0 away from some training example. If this happens your implementation should handle it gracefully and return the appropriate class label.

When you think you're done, execute the following cell to run three final unit tests corresponding to the example on Slide 21 of the KNN Lecture slides on Canvas. Make sure that the changes you make in **Part C** do not affect the unit tests from **Part B**.

```
In [ ]: class KNN:
        """
        Class to store data for classification problems
        """
        def __init__(self, X_train, y_train, K=5, distance_weighted=False):
            """
            Creates a kNN instance

            :param X_train: Training data input in 2D ndarray
```

```

:param y_train: Training data output in 1D ndarray
:param K: The number of nearest points to consider in classification
:param distance_weighted: Bool indicating whether to use distance weighting
"""

from sklearn.neighbors import BallTree
self.balltree = BallTree(X_train)

self.y_train = y_train
self.K = K
self.distance_weighted = distance_weighted

def majority(self, neighbor_indices, neighbor_distances=None):
    """
    Given indices of nearest neighbors in training set, return the majority label.
    Break ties by considering 1 fewer neighbor until a clear winner is found.

    :param neighbor_indices: The indices of the K nearest neighbors in self.X_train
    :param neighbor_distances: Corresponding distances from query point to K nearest neighbors.
    """

    # If only one neighbor is left, return its label.
    if len(neighbor_indices) == 1:
        return self.y_train[neighbor_indices[0]]

    # If distances are not provided, use equal distances/weights.
    if neighbor_distances is None:
        neighbor_distances = np.ones(len(neighbor_indices))

    # Get the labels for the nearest neighbors.
    neighbor_labels = self.y_train[neighbor_indices]

    # Part C: distance-weighted KNN
    if self.distance_weighted:

        # If any neighbor has distance 0, the query point is exactly on top
        # of one or more training points. In that case, use only those exact
        # matches and return their majority label.
        zero_distance_indices = neighbor_indices[neighbor_distances == 0]

        if len(zero_distance_indices) > 0:
            return self.majority(zero_distance_indices)

        # Compute inverse-distance weights.
        weights = 1 / neighbor_distances

        # Sum weights separately for each label.
        weighted_votes = {}

        for label, weight in zip(neighbor_labels, weights):
            if label not in weighted_votes:
                weighted_votes[label] = 0

```

```

        weighted_votes[label] += weight

    # Find the largest total weighted vote.
    max_vote = max(weighted_votes.values())

    # Find all labels tied for the largest weighted vote.
    winners = [
        label for label, vote in weighted_votes.items()
        if vote == max_vote
    ]

    # If there is one winner, return it.
    if len(winners) == 1:
        return winners[0]

    # If there is a tie, remove the farthest neighbor and try again.
    return self.majority(neighbor_indices[:-1], neighbor_distances[:-1])

# Part B: unweighted KNN
# Count how many times each label appears.
labels, counts = np.unique(neighbor_labels, return_counts=True)

# Find the largest vote count.
max_count = np.max(counts)

# Find all labels tied for the largest vote count.
winners = labels[counts == max_count]

# If there is one winner, return it.
if len(winners) == 1:
    return winners[0]

# If there is a tie, remove the furthest neighbor and try again.
return self.majority(neighbor_indices[:-1], neighbor_distances[:-1])

def classify(self, x):
    """
    Given a query point, return the predicted label

    :param x: a query point stored as an ndarray
    """

    # Query the BallTree for the K nearest neighbors.
    distances, indices = self.balltree.query(
        x.reshape(1, -1),
        k=self.K
    )

    # Extract the results for this single query point.
    neighbor_distances = distances[0]
    neighbor_indices = indices[0]

```

```

        # Use majority voting or distance-weighted voting.
        return self.majority(neighbor_indices, neighbor_distances)

    def predict(self, X):
        """
        Given an ndarray of query points, return yhat, an ndarray of predictions

        :param X: an (m x p) dimension ndarray of points to predict labels for
        """

        # Classify every row in X.
        predictions = [self.classify(x) for x in X]

        # Return predictions as a numpy array.
        return np.array(predictions)

```

```

In [ ]: from tests import tests
        tests.run_test_suite('prob 2B', KNN)

```

```

test5NNclassify (tests.tests.TestWeightedKNN.test5NNclassify)
test 5NN ... ok
test3NNclassify (tests.tests.TestWeightedKNN.test3NNclassify)
test 3NN. Checks divide-by-zero issue. ... ok
test5NNpredict (tests.tests.TestWeightedKNN.test5NNpredict)
test 5NN prediction ... ok

```

```

-----
Ran 3 tests in 0.027s

```

```

OK

```

Part D [8 points]: Use your `KNN` class to perform weighted KNN on the validation data with $K = 100$ and do the following:

[4 points] Create a **confusion matrix** (feel free to use the Scikit-Learn [confusion_matrix](#) function).

```

In [23]: from sklearn.metrics import confusion_matrix
        knn = KNN(X_train.to_numpy(), y_train.to_numpy(), K=100, distance_weighted=True)
        yhat_valid = knn.predict(X_val.to_numpy())

        # Choose the label order for the confusion matrix.
        labels = ["THEFT", "BATTERY", "CRIMINAL DAMAGE", "ASSAULT"]

        # Create the confusion matrix.
        cm = confusion_matrix(y_val.to_numpy(), yhat_valid, labels=labels)

        # Put the confusion matrix into a DataFrame so it is easier to read.
        cm_df = pd.DataFrame(
            cm,
            index=labels,
            columns=labels
        )

```

```
# Display the confusion matrix.
cm_df
```

Out[23]:

	THEFT	BATTERY	CRIMINAL DAMAGE	ASSAULT
THEFT	4307	2253	753	513
BATTERY	1942	4036	1139	1015
CRIMINAL DAMAGE	1367	2021	1023	564
ASSAULT	984	1763	580	678

[4 points] Based on your confusion matrix, which crime types are most frequently misclassified as other crime types?

Based on the confusion matrix, the most frequent misclassification is THEFT being classified as BATTERY, with 2253 theft cases predicted as battery.

Part E [12 points]: [4 points] Create a plot of the accuracy of both Unweighted and Distance-Weighted KNN on the validation set on the same set of axes for various values of K . Your plot should show the approximate value of K that maximizes the accuracy. Note that accuracy will be below 50%, but above 25% (i.e., better than random classification).

```
In [21]: acc = []
wacc = []
allks = [50, 100, 150, 200, 250, 300]

for k in allks:
    # Unweighted KNN for this value of k.
    knn = KNN(
        X_train.to_numpy(),
        y_train.to_numpy(),
        K=k,
        distance_weighted=False
    )

    # Predict validation labels using unweighted KNN.
    yhat_valid = knn.predict(X_val.to_numpy())

    # Store unweighted accuracy.
    acc.append(accuracy_score(y_val.to_numpy(), yhat_valid))

    # Distance-weighted KNN for this value of k.
    wknn = KNN(
        X_train.to_numpy(),
        y_train.to_numpy(),
        K=k,
        distance_weighted=True
    )

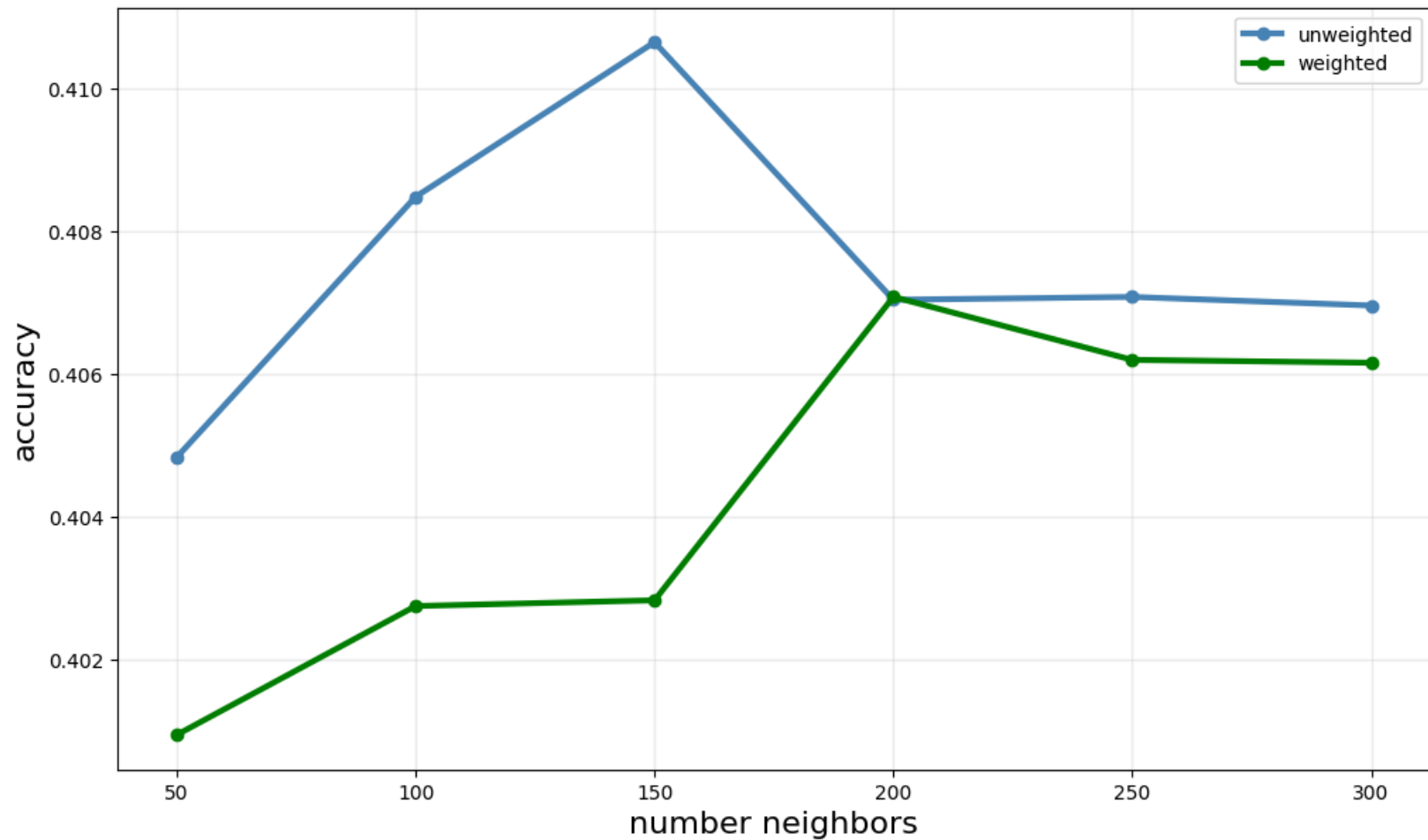
    # Predict validation labels using weighted KNN.
    wyhat_valid = wknn.predict(X_val.to_numpy())
```

```

# Store weighted accuracy.
wacc.append(accuracy_score(y_val.to_numpy(), wyhat_valid))

fig, ax = plt.subplots(nrows=1,ncols=1,figsize=(12, 7))
ax.plot(allks, acc, marker="o", color="steelblue", lw=3, label="unweighted")
ax.plot(allks, wacc, marker="o", color="green", lw=3, label="weighted")
ax.set_xlabel("number neighbors", fontsize=16)
ax.set_ylabel("accuracy", fontsize=16)
ax.legend(loc="upper right")
plt.xticks(range(50, 301, 50))
ax.grid(alpha=0.25)

```



[4 points] Based on the plot, answer the following questions:

- For general K , does Unweighted or Weighted KNN appear to perform better?

- Which value of K attains the best accuracy on the validation set?

Open questions: Why do you think this is the case? How can you explain this?

Unweighted KNN generally performs better than Distance-Weighted KNN for most values of (K) . The highest validation accuracy appears to occur for Unweighted KNN with $(K = 150)$, where the accuracy is a little above (0.410) . Weighted KNN improves as (K) increases and is close to unweighted KNN around $(K = 200)$, but it does not reach the best accuracy from the unweighted model.

This may happen because crime type based only on latitude and longitude is noisy. A very close nearby incident is not always more informative than the broader neighborhood pattern, so giving too much weight to the closest points can hurt performance. Unweighted KNN with a moderately large (K) smooths over local noise and captures a more stable pattern in the area. However, if (K) gets too large, the model may include too many less relevant points, which explains why accuracy stops improving after around $(K = 150)$.

[4 points] If you had unrestricted access to data, how might you improve the model to try to increase classification accuracy?

I would add more features beyond just latitude and longitude such as the date and time of incident, neighborhood, if it was inside or outside. I would also increase the years of crime data to give the model more examples.

Acknowledgment: Noah Smith, Chris Ketelsen, Chenhao Tan